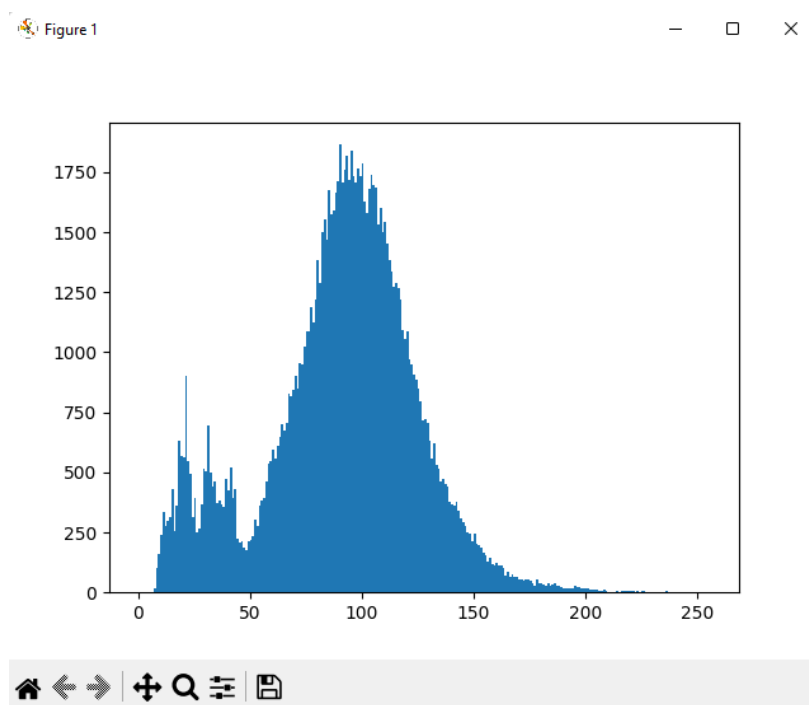


SOURCE CODE :

```
import cv2  
  
import matplotlib.pyplot as plt  
  
img=cv2.imread("leaf.jpg",0)  
img=cv2.resize(img,(300,400))  
cv2.imshow("Image",img)  
plt.hist(img.ravel(),256,[0,256])  
plt.show()  
cv2.imwrite('C:/Users/Lenovo/Desktop/newimg.jpg',img)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

OUTPUT :



SOURCE CODE :

```
import cv2

img=cv2.imread('black.jpg',cv2.IMREAD_COLOR)

Value=img[10,10,:]

print("Accessing Pixel Values :",Value)

for j in range (10,50):

    for i in range(11,55):

        img[j,i,0]=255

Value=img[10,11,:]

print("Modifying Pixel",Value)

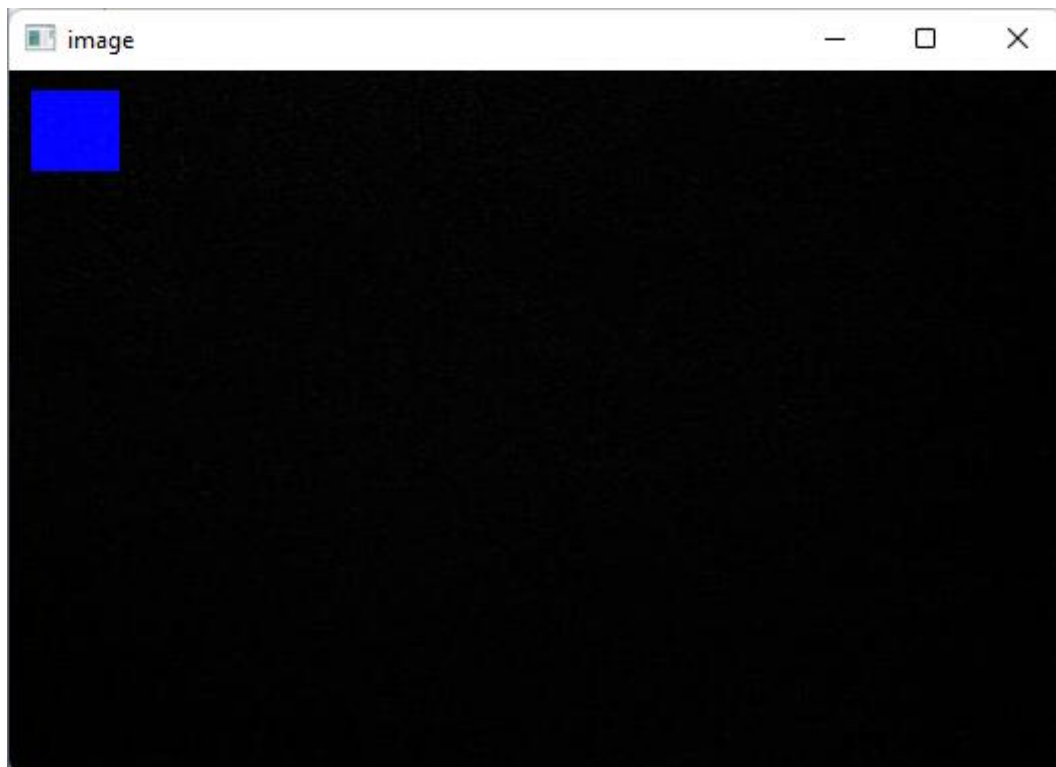
cv2.imshow("image",img)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

OUTPUT :

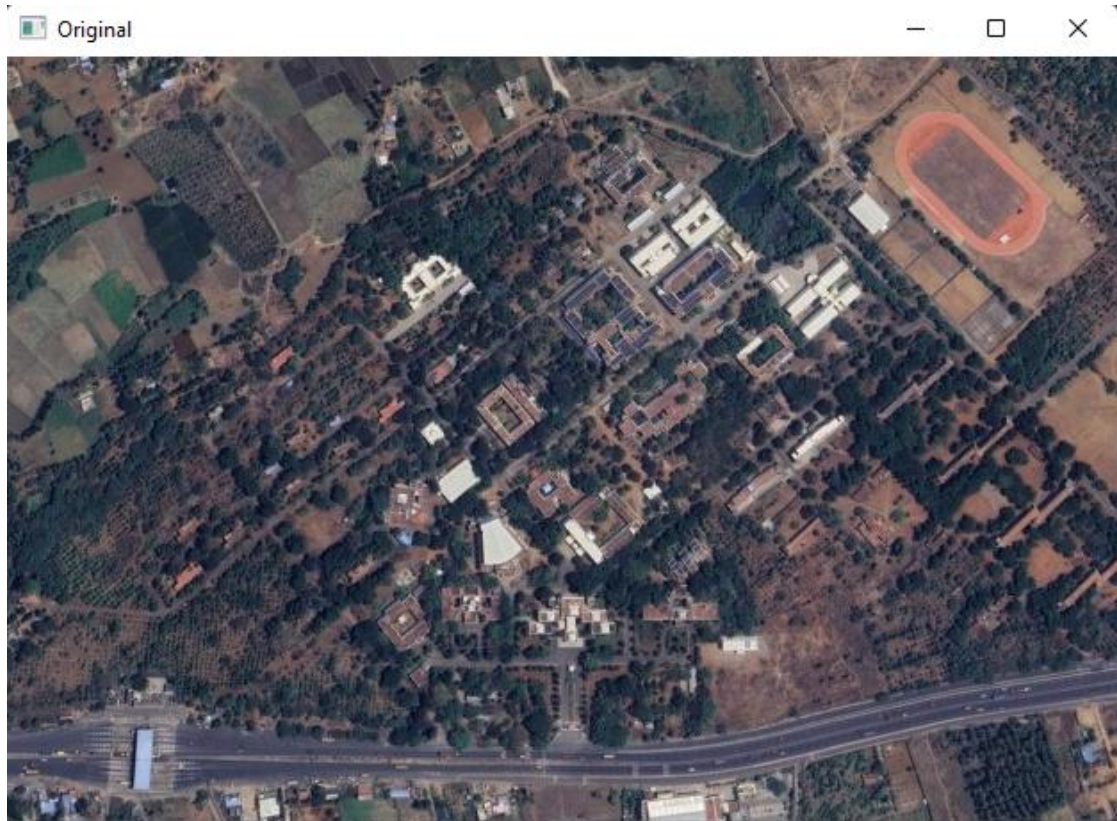
```
*IDLE Shell 3.10.6*
File Edit Shell Debug Options Window Help
Python 3.10.6 (tags/v3.10.6:9c7b4bd, Aug 1 2022, 21:53:49) [MSC v.1932 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\SAM\Desktop\cv\cvex02.py =====
Accessing Pixel Values : [5 5 5]
Modifying Pixel [255 5 5]
```



SOURCE CODE :

```
import cv2  
img=cv2.imread('sat.jpg')  
r=cv2.resize(img,(500,350))  
rot=cv2.getRotationMatrix2D((img.shape[1]/2,img.shape[0]/2),30,1)  
rotated=cv2.warpAffine(img,rot,(img.shape[1],img.shape[0]))  
cv2.imshow('Original',img)  
cv2.imshow('Rised',r)  
cv2.imshow('rotated',rotated)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

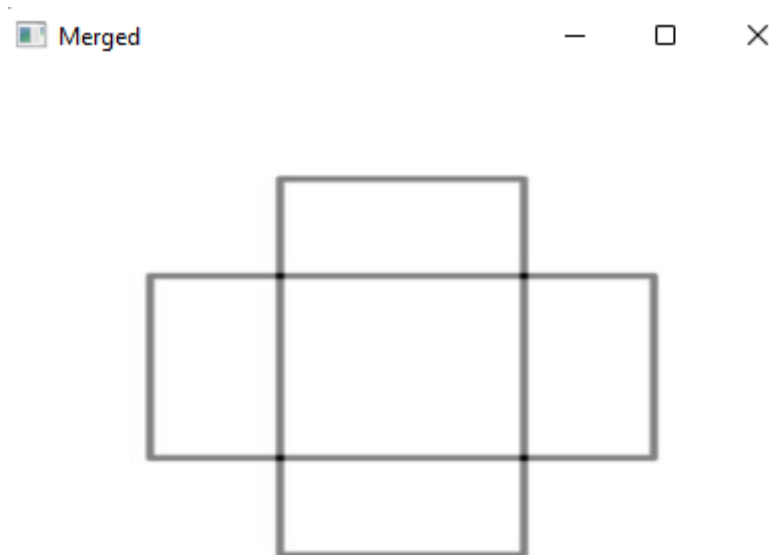
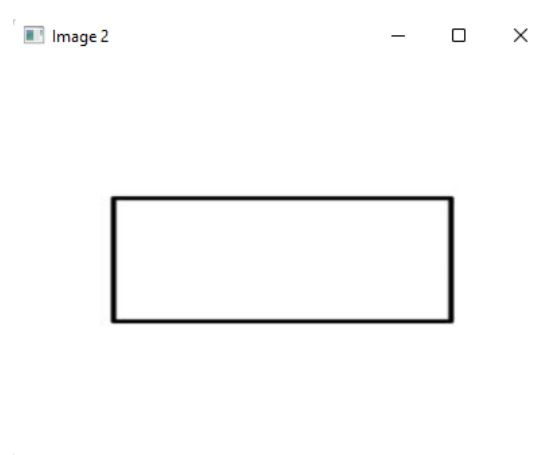
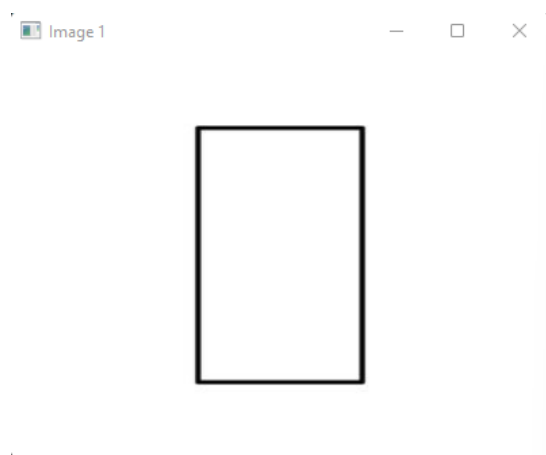
OUTPUT :



SOURCE CODE :

```
import cv2  
img1=cv2.imread("rect1.jpg")  
img2=cv2.imread("rect2.jpg")  
img1=cv2.resize(img1,(400,300))  
img2=cv2.resize(img2,(400,300))  
res=cv2.addWeighted(img1,0.5,img2,0.5,0)  
cv2.imshow("Image 1",img1)  
cv2.imshow("Image 2",img2)  
cv2.imshow("Merged",res)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

OUTPUT:



SOURCE CODE :

```
import cv2
import numpy as np
img1=cv2.imread("img.jpg")
img2=cv2.imread("img.jpg")
mean=cv2.blur(img1,(9,9))
gaussian=cv2.GaussianBlur(img2,(9,9),0)
cv2.imshow("Mean blur",np.hstack((img1,mean)))
cv2.imshow("Gaussian blur",np.hstack((img2,gaussian)))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT :

Mean blur



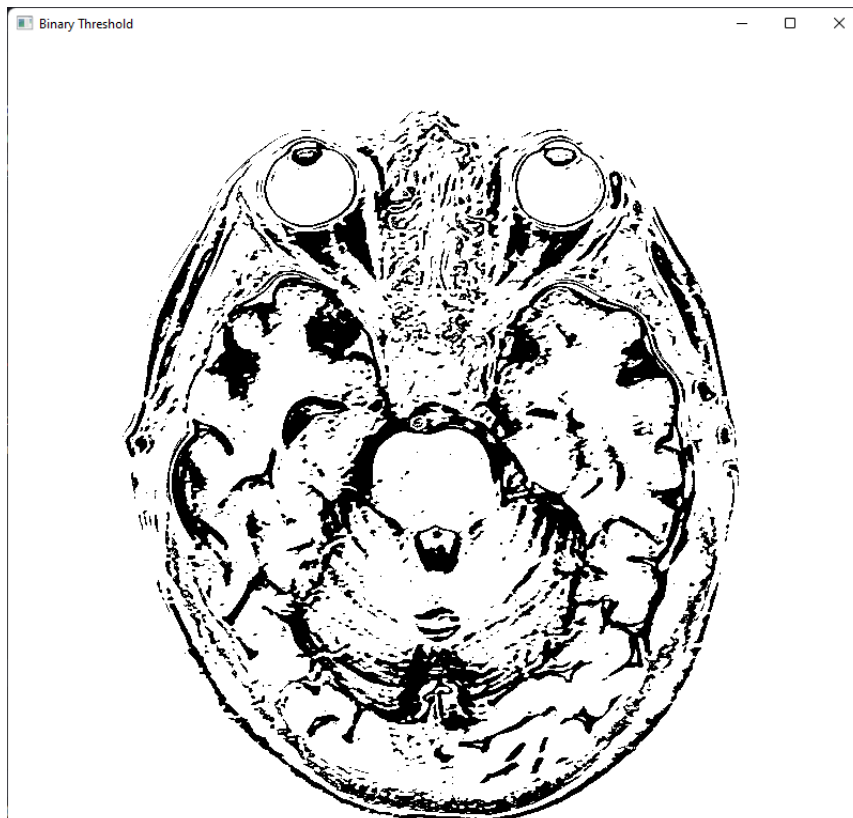
Gaussian Blur Image



SOURCE CODE :

```
import cv2  
img=cv2.imread("mri.jpg")  
img=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)  
ret,thresh=cv2.threshold(img,120,455,cv2.THRESH_BINARY)  
cv2.imshow("Original Image",img)  
cv2.imshow("Binary Threshold",thresh)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

OUTPUT :



SOURCE CODE :

```
import cv2  
img=cv2.imread("sud.png",cv2.IMREAD_GRAYSCALE)  
sobelx=cv2.Sobel(img,cv2.CV_64F,1,0,ksize=3)  
sobely=cv2.Sobel(img,cv2.CV_64F,0,1,ksize=3)  
scharrx=cv2.Scharr(img,cv2.CV_64F,1,0)  
scharry=cv2.Scharr(img,cv2.CV_64F,0,1)  
cv2.imshow("ORIGINAL",img)  
cv2.imshow('Sobel X',sobelx)  
cv2.imshow('Sobel Y',sobely)  
cv2.imshow("Scharr X",scharrx)  
cv2.imshow("Scharr Y",scharry)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

OUTPUT :

ORIGINAL

	2		6	4		3	7	5	1	2	9
			1		3	9	6	4		5	
3	6	4	9	7	5						4
6	9		2	1	4	5	8	7			6
1		2		5	7	4	9	6		1	
4	7	5	8	6	9		3	2		4	7
2	1		5	8	6		4		2		1
7			4	9	1		5		6	7	3
5		8	7	3		6	1	9		8	5
3	2	4	1		8	9	6	5			4
	6	1	9		5		7	4		6	8
	5	7		4		8	2				9

Sobel X

	2		6	4		3	/	5	1	2	9
			1		3	9	6	4		5	
3	6	4	9	/	5						4
6	9		2	1	4	5	8	/			6
1		2		5	/	4	9	6		1	
4	/	5	8	6	9		3	2		4	/
2	1		5	8	6		4		2		1
/			4	9	1		5		6	/	3
5		8	/	3		6	1	9		8	5
3	2	4	1		8	9	6	5			4
	6	1	9		5		/	4		6	8
	5	/		4		8	2				9

Sobel Y

	2		6	4		3	7	5	1	2	9
			1		3	9	6	4		5	
3	6	4	9	7	5						4
6	9		2	1	4	5	8	7			6
1		2		5	7	4	9	6		1	
4	7	5	8	6	9		3	2		4	7
2	1		5	8	6		4		2		1
7		4	9	1		5		6	7	3	
5		8	7	3		6	1	9		8	5
3	2	4	1		8	9	6	5			4
	6	1	9		5		7	4		6	8
	5	7		4		8	2				9

Scharr X

	2		6	4		3	/	5	1	2	9
			1		3	9	6	4		5	
3	6	4	9	/	5						4
6	9		2	1	4	5	8	/			6
1		2		5	/	4	9	6		1	
4	/	5	8	6	9		3	2		4	/
2	1		5	8	6		4		2		1
/			4	9	1		5		6	/	3
5		8	/	3		6	1	9		8	5
3	2	4	1		8	9	6	5			4
	6	1	9		5		/	4		6	8
	5	/		4		8	2				9

Scharr Y

	2		6	4		3	7	5	1	2	9
			1		3	9	6	4		5	
3	6	4	9	7	5						4
6	9		2	1	4	5	8	7			6
1		2		5	7	4	9	6		1	
4	7	5	8	6	9		3	2		4	7
2	1		5	8	6		4		2		1
7		4	9	1		5		6	7	3	
5		8	7	3		6	1	9		8	5
3	2	4	1		8	9	6	5			4
	6	1	9		5		7	4		6	8
	5	7		4		8	2				9

SOURCE CODE :

```
import cv2

import matplotlib.pyplot as plt

imgObj=cv2.imread('img.jpg')

plt.axis("off")

plt.title("Original Image")

plt.imshow(cv2.cvtColor(imgObj,cv2.COLOR_BGR2RGB))

plt.show()

blue=cv2.calcHist([imgObj],[0],None,[256],[0,256])

red=cv2.calcHist([imgObj],[1],None,[256],[0,256])

green=cv2.calcHist([imgObj],[2],None,[256],[0,256])

plt.title("Histogram of RGB Image")

plt.hist(blue,color='blue')

plt.hist(red,color='red')

plt.hist(green,color='green')

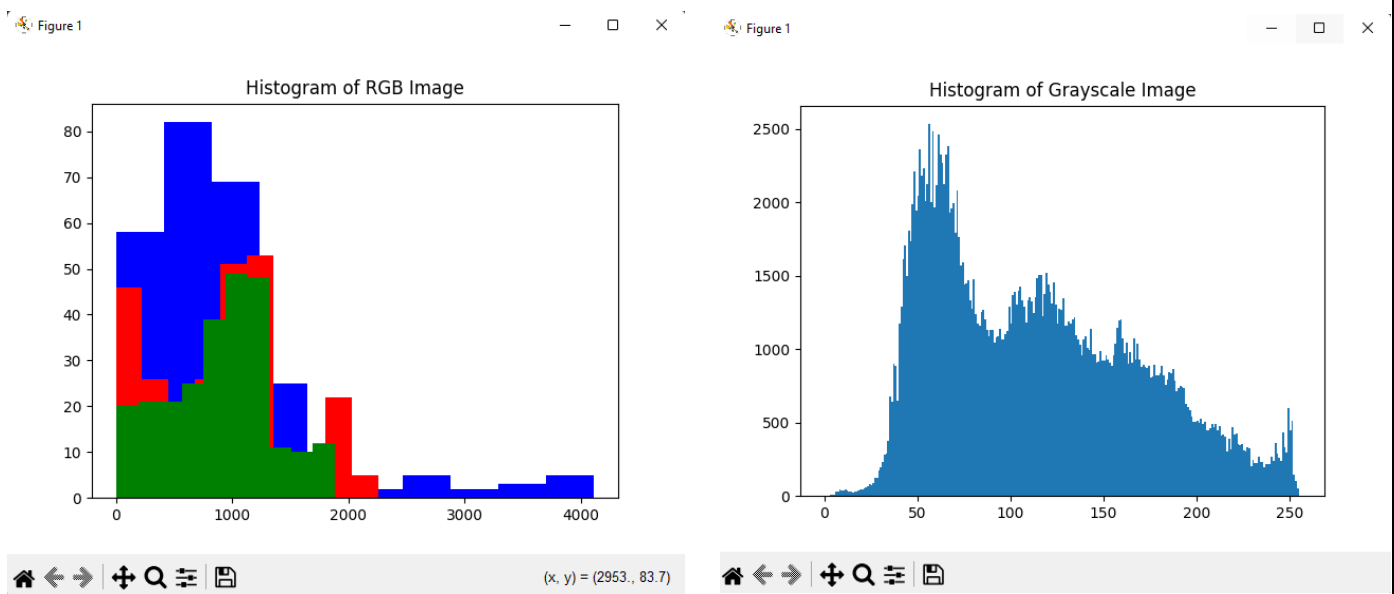
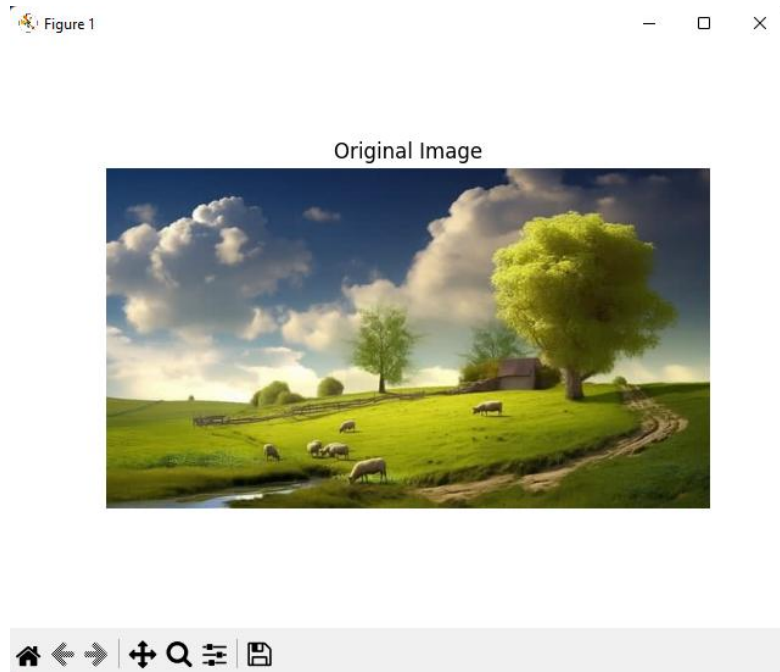
plt.show()

plt.title("Histogram of Grayscale Image")

plt.hist((cv2.cvtColor(imgObj,cv2.COLOR_BGR2GRAY)).ravel(),256,[0,256])

plt.show()
```

OUTPUT :

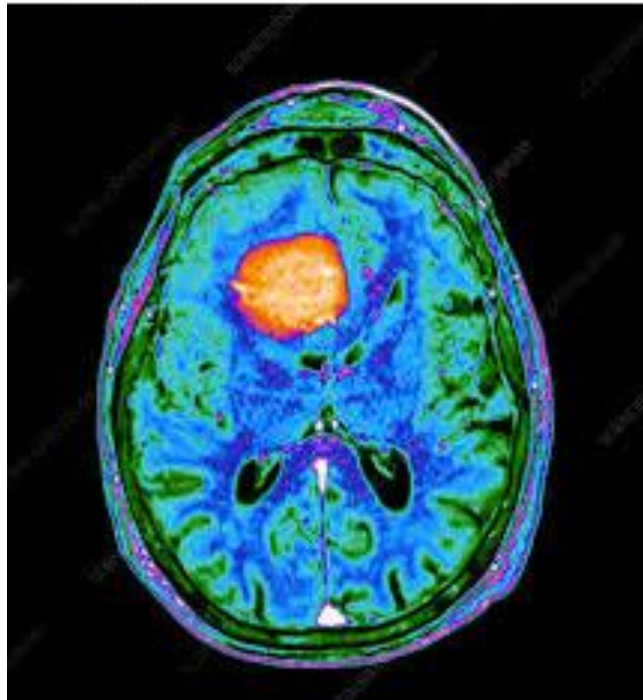


SOURCE CODE :

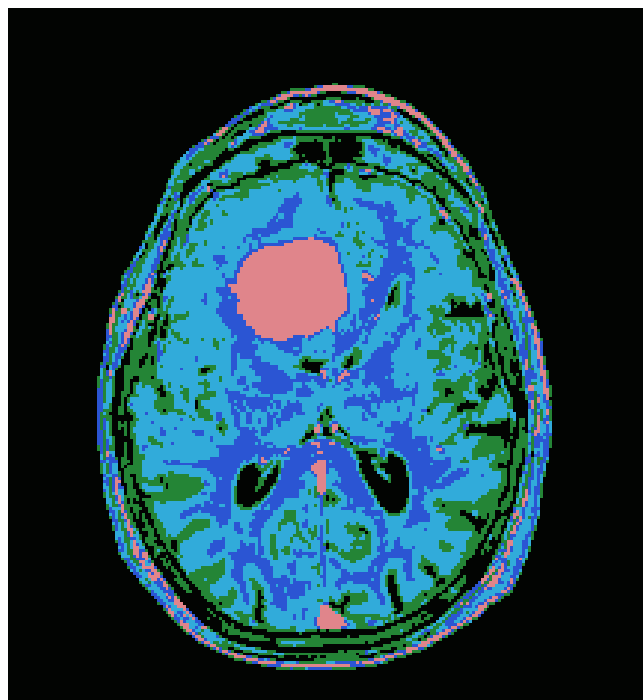
```
import numpy as np
import cv2
img=cv2.imread('try.jfif')
z=img.reshape((-1,3))
z=np.float32(z)
criteria=(cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_MAX_ITER,10,1.0)
K=5
ret,label,center=cv2.kmeans(z,K,None,criteria,10,cv2.KMEANS_RANDOM_CENTERS)
center=np.uint8(center)
res=center[label.flatten()]
res2=res.reshape((img.shape))
cv2.imshow('Image using K- Means Cluster',res2)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT :

 Original



 K- Means



SOURCE CODE :

```
import cv2
import os
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

data = []
labels = []
dataset_path = "dataset"

# Load images
for label in os.listdir(dataset_path):
    folder = os.path.join(dataset_path, label)
    for img_name in os.listdir(folder):
        img_path = os.path.join(folder, img_name)
        img = cv2.imread(img_path)
        img = cv2.resize(img, (50,50))
        img = img.flatten()
        data.append(img)
        labels.append(label)

data = np.array(data)

knn = KNeighborsClassifier(n_neighbors=3) #Train KNN
knn.fit(data, labels)

test_img = cv2.imread("test-img.jpg") #Test KNN
```

```
cv2.imshow("Test-Image",test_img)

test_img = cv2.resize(test_img, (50,50))

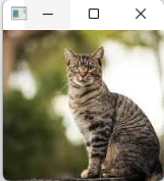
test_img = test_img.flatten().reshape(1,-1)

prediction = knn.predict(test_img)

print("Predicted Class:", prediction[0])
```

OUTPUT :

```
IDLE Shell 3.10.6
File Edit Shell Debug Options Window Help
Python 3.10.6 (tags/v3.10.6:9c7b4bd, Aug 1 2022, 21:53:49) [MSC v.1932 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/SAM/Desktop/cv/EX10TRY.py =====
>>> Predicted Class: cat
```



```
IDLE Shell 3.10.6
File Edit Shell Debug Options Window Help
Python 3.10.6 (tags/v3.10.6:9c7b4bd, Aug 1 2022, 21:53:49) [MSC v.1932 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/SAM/Desktop/cv/EX10TRY.py =====
>>> Predicted Class: cat
>>>
===== RESTART: C:/Users/SAM/Desktop/cv/EX10TRY.py =====
>>> Predicted Class: dog
```

